

Topic 3: CSS

Standard practice

By default:

box-sizing: content-box;

means width = content only (padding & border added later)

```
* {  
  box-sizing: border-box;  
}
```

Now: width includes padding + border

This prevents 90% of layout miscalculations.

CSS Priority

Inline Styles > ID > Class > Tag

FLEXBOX

Flexbox works on **AXES**, not directions.

Property

justify-content

align-items

Controls

Main axis

Cross axis

For default flex JC== horizontal AL==vertical

For Column flex JC==Vertical AL==horizontal

POSITIONING

- 1 Does this element affect other elements? (*Flow vs Out-of-flow*)
- 2 If moved, what is its reference system? (*Which box does it measure from?*)

1. position: static (Default Reality)

- Element stays in **Normal Document Flow**
- Ignores top, left, etc.
- Browser lays it out using:
 - Block formatting
 - Inline formatting

Key Insight:

 Static elements are **not positioned** — they are **computed**.

2. position: relative — “Shift Without Leaving Reality”

Element **stays in flow**

You can **visually move it**

Original space is **still reserved**

The browser:

- 1 Lays the element normally
- 2 Then applies a **visual offset layer**

It's like:

“Render it normally... now shift the paint.”

IMP

Because relative creates a Positioning Context for absolutely positioned children.

3. position: absolute — Leaves the Flow Completely

Behavior:

- Removed from document flow
- Other elements behave like it doesn't exist
- Positioned relative to:

Nearest ancestor that is NOT static

If none exists → relative to **viewport**.

```
.parent {  
  position: relative;  
}
```

```
.child {  
  position: absolute;  
  top: 0;  
  right: 0;  
}
```

The Most Important Rule Beginners Miss

If no positioned ancestor exists:

position: absolute;

= behaves like:

position: absolute to <html>

This is why layouts "randomly break."

4. position: fixed — Anchored to the Viewport

Behaviour:

- Removed from flow
- Ignores parents
- Always relative to **viewport**
- Does NOT move on scroll

Z-Index Only Works on Positioned Elements

z-index: 10;

for stacking elements on top

- ✓ Flexbox → for alignment
- ✓ Grid → for structure
- ✓ Position → for **overlays & relationships**

Use Positioning for **UI attachment**, not layout.

Media Query

✅ Mobile-First Approach (Industry Standard)

Start with mobile styles, then scale up.

```
/* Default = Mobile */
```

```
.container {  
  padding: 10px;  
}
```

```
/* Tablet */
```

```
@media (min-width: 768px) {  
  .container {  
    padding: 20px;  
  }  
}
```

```
/* Desktop */
```

```
@media (min-width: 1024px) {  
  .container {  
    padding: 40px;  
  }  
}
```

Min-width === above that width apply this style

```
@media (min-width: 768px) { }
```

Means

Apply when screen is 768px OR BIGGER

Max-width=== till that width apply this style

```
@media (max-width: 768px) { }
```

Means:

Apply when screen is 768px OR SMALLER

3. Breakpoints You MUST Know (Industry Reality)

Do NOT invent random values. Use meaningful breakpoints.

Device	Width
Mobile Small	max-width: 480px
Mobile	max-width: 767px
Tablet	768px – 1023px
Laptop	1024px – 1279px
Desktop	1280px+

```
@media (min-width: 768px) and (max-width: 1024px) {  
  body {  
    background: lightblue;  
  }  
}
```

GRID

```
.container {  
  display: grid;  
  grid-template-columns: 200px 200px 200px;  
  grid-template-rows: 100px 100px;  
}
```

You just created:

3 columns × 2 rows

GTC FOR WIDTH

GTR FOR HEIGHT

fr = fraction of available space

grid-template-columns: 1fr 1fr 1fr

3 equally width colum

grid-template-columns: 2fr 1fr;

First column is twice as wide.

grid-template-columns: repeat(4, 1fr);

GAP IN GRID

gap: 20px;

Or:

```
row-gap: 10px;  
column-gap: 30px;
```

GRID-TEMPLATE AREA

Step 1 — Create a Grid Container

First, apply `display: grid` to the parent.

```
.container {  
  display: grid;  
}
```

Step 2 — Define Rows & Columns

You must define structure using:

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 3fr;  
  grid-template-rows: auto 1fr auto;  
}
```

Step 3 — Draw Layout Using `grid-template-areas`

Now we **name each section visually**.

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 3fr;  
  grid-template-rows: auto 1fr auto;
```

```
  grid-template-areas:
```

```
    "header header"
```

```
    "sidebar main"
```



```
"footer footer";  
}
```

Each string = one row
Each word = one column

Step 4 — Assign Elements to Those Areas

Now connect HTML elements to the named areas using grid-area.

```
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
.main { grid-area: main; }  
.footer { grid-area: footer; }
```

Step 5 — Write Clean HTML (No Positioning Needed!)

```
<div class="container">  
  <div class="header">Header</div>  
  <div class="sidebar">Sidebar</div>  
  <div class="main">Main Content</div>  
  <div class="footer">Footer</div>  
</div>
```